

DSI - Teorico

#Estudio

#CC

#Diseño

Unidad 1

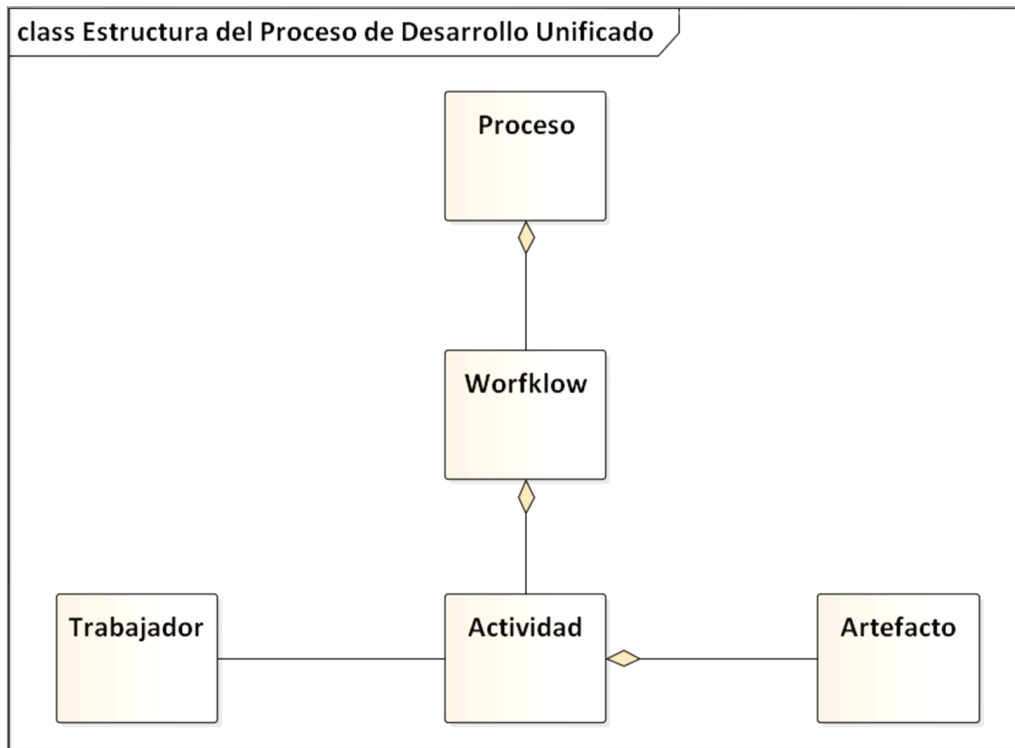
PUD

El Proceso Unificado de Desarrollo es un proceso definido en un entorno de proceso genérico (framework) el cual Puede especializarse para una gran variedad de sistemas de software, en diversas áreas de aplicación.

Del Proceso de Desarrollo se espera que sea capaz de evolucionar. Durante su evolución se limita a las realidades que se imponen.

Los pilares en el Desarrollo de Software se pueden describir como un proyecto que se adapta a un proceso el cual es automatizado con herramientas, en donde se incorporan personas y se obtiene como resultado un producto o servicio.

El Proceso Unificado es un entorno de Proceso que permite una variedad de ciclos de vida, selecciona que artefactos producir, define actividades y trabajadores y modela conceptos



Actividad: Una unidad de trabajo

Trabajador: Un rol desempeñado por un individuo o equipo

Artefacto: Una pieza de información que es producida, modificada o usada por un proceso

Aspectos Característicos del PUD

- Ciclo de vida Iterativo e Incremental

Con workflows de:

Requerimientos

Análisis-Diseño

Implementación

Prueba

Con fases de:

Inicio: Define el alcance del proyecto y desarrolla el caso de negocio

Elaboración: Planifica el Proyecto, especifica aspectos, y delinea la arquitectura

Construcción: Construye el producto

Transición: Transfiere el producto a sus usuarios

Una iteración es una secuencia de actividades con un plan establecido y un criterio de evaluación, que resulta en un release ejecutable.

- Conducido por Casos de Uso

Los casos de uso conducen el Proceso de Iteración. Cada caso de uso captura el Valor de los Requerimientos, conduce el Proceso y delinea la arquitectura

- Centrado en la Arquitectura

Se divide en 4 vistas:

- Vista de Diseño: vocabulario, funcionalidad
- Vista de Proceso: performance, escalabilidad
- Vista de Implementación: montaje del sistema, administración de la configuración
- Vista de Despliegue: topología, distribución, entrega e instalación

Los diagramas de UML construyen las vistas en cada modelo, cada workflow está asociado con uno o más modelos.

Workflow de Requerimientos

Propósito:

Desarrollar un modelo del sistema que se va a construir.

Utiliza casos de uso para construir el modelo. Los requerimientos funcionales se estructura de manera natural en casos de uso. La mayoría de los requerimientos no funcionales pueden tratarse en el contexto de un casos de uso.

- Fase de Comienzo/Inicio: identificar la mayoría de los use cases para delimitar el sistema y el alcance del proyecto.
- Detallar los más críticos: Fase de Elaboración capturar la mayoría de los requerimientos restantes para poder estimar el esfuerzo de desarrollo.
- Fase de Construcción: identificar e implementar los requerimientos restantes

- Fase de Transición: casi no hay captura de requerimientos, a menos que se requieran cambios.

Workflow de Análisis

Propósito:

Permite una especificación más precisa de los requerimientos.

Se describe usando el lenguaje del desarrollador.

Estructura los requerimientos.

Puede verse como un primer corte del modelo de diseño.

- Fase de Comienzo: Contribuye a planificación de los incrementos.
- Fases de Elaboración y Construcción: Contribuye a obtener una estructura robusta y estable y facilita una comprensión más profunda de los requerimientos

Workflow de Diseño

Propósito:

Adquirir una comprensión en profundidad de los aspectos relacionados con los requerimientos no funcionales y restricciones tecnológicas.

Crear una entrada para el workflow de implementación.

Descomponer los trabajos de implementación en partes manejables y permitiendo la concurrencia entre equipos de desarrollo.

Capturar interfaces entre subsistemas.

Crear una abstracción de la solución de diseño sin estar restringidos a una tecnología.

Fase de Elaboración:

- Contribuye a una arquitectura estable y robusta
- Fase de Construcción
- Crear un plano para la implementación

UML

UML es un lenguaje de modelado, de propósito general, usado para la visualización, especificación, construcción y documentación de sistemas Orientados a Objetos

UML tiene reglas semánticas para:

- Nombres: cómo llamar los elementos, relaciones y diagramas.
- Alcance: contexto que da un significado específico al nombre.
- Visibilidad: cómo se pueden ver y usar esos elementos por otros.
- Integridad: cómo se relacionan apropiada y consistentemente unos elementos con otros.
- Ejecución: que significa ejecutar o simular un modelo dinámico.

Elementos estructurales:

- Clases
- Interfaces
- Casos de uso
- Colaboraciones
- Clases activas
- Componentes
- Nodos

Diagramas de Comportamiento: muestra aspectos de comportamiento de un sistema.

Diagrama de Comunicación

Clasificación: De comportamiento, dinámico, lógico.

Sirve para:

- Proveer una vista de la colaboración de objetos en un ambiente de tiempo real.
- Distribuir funcionalidad en las clases, explorando los aspectos comportamentales de un sistema.
- Modelar la implementación lógica de una operación compleja, particularmente la que interactúa con muchos objetos.

Diagrama de secuencia que enfatiza la organización estructural de los objetos. Muestra un conjunto de objetos y conexiones entre esos objetos y los mensajes enviados y recibidos por esos objetos.

Diagrama de Secuencia

Clasificación: De comportamiento, dinámico, lógico.

Sirve para:

- Validar y describir la lógica de un escenario.
- Explorar el diseño controlando la invocación de las operaciones definidas en las clases.
- Detectar cuellos de botella en un diseño orientado a objetos.
- Analizar qué clases son complejas en el sistema.

Diagrama de secuencia que enfatiza el orden de los mensajes en función del tiempo. Muestra un conjunto de objetos y los mensajes enviados y recibidos por esos objetos.

Diagrama de Máquina de Estados

Clasificación: De comportamiento, dinámico, lógico.

Sirve para:

- Explorar el comportamiento complejo de una clase, actor, subsistema o componente.
- Modelar sistemas de tiempo real.

Muestra una máquina de estados, compuesta por estados, transiciones, eventos y actividades.

Diagrama de Actividad

Clasificación: De comportamiento, dinámico, lógico.

Sirve para Mostrar:

- Una operación compleja.
- Una regla de negocio compleja.
- Un caso de uso.
- Algunos casos de uso.
- Un proceso de negocio.
- Procesos de software.

Muestra el conjunto de actividades, y el flujo de secuencia o desglose de actividad en actividad, y los objetos que actúan y que son afectados.

Diagramas de Interacción: subtipo de los de comportamiento que enfatiza interacción entre objetos.

Diagramas de Estructura: Muestra elementos de especificación, independientes del tiempo

- Diagrama de paquetes

Clasificación: De estructura, estático, lógico.

Sirve para:

- Descripción a alto nivel de requerimientos
- Descripción a alto nivel del diseño
- Modularizar diagramas complejos
- Organizar código

Muestra un conjunto de paquetes y sus relaciones

- Diagrama de clases

Clasificación: De estructura, estático, lógico.

Sirve para:

- Explorar conceptos del dominio
- Analizar requerimientos
- Mostrar el diseño detallado de software orientado a objetos

Muestra un conjunto de clases, interfaces y colaboraciones y sus relaciones

Fase de análisis

En la fase de análisis se realiza el descubrimiento de comportamiento, es decir, se completan las clases con los métodos necesarios para que el sistema haga para lo que fue diseñado.

Los artefactos que se producen en esta etapa son:

- Clases de análisis
- Realizaciones de C.U
- Paquetes de análisis
- Descripción de arquitectura

Modelo de análisis

El modelo de analisis se produce como salida del workflow de analisis (el cual utiliza la entrada del modelo de requerimientos). El modelo de analisis se constituye por:

- Realizaciones de casos de uso de analisis
 - Vista estatica
 - La cual representa un instante especifico del sistema
 - Diagrama de clases
 - Vista dinamica
 - Diagrama de comunicacion
 - Diagrama de secuencia
- Clases de analisis
 - Si una clase de analisis esta relacionada con otra clase, de tipo Estado, no necesariamente hace falta un atributo estado para que exista esta relacion, puede haber otro atributo (la fecha) que puede definir el estado de esta clase.
 - Diagrama de clases
 - Diagrama de maquina de estados
 - Las maquinas de estado varian en funcion de su comportamiento y esto se ve reflejado en el estado
- Paquetes de analisis
 - Diagrama de paquetes
- Descripcion de arquitectura

POO

Un objeto representa un elemento, unidad o entidad individual e identificable, ya sea real o abstracta, con un rol bien definido en el dominio del problema.

Una clase describe un grupo de objetos que comparten una estructura y un comportamiento comunes y es una definición de un conjunto de objetos que comparten una estructura común y un comportamiento común.

Los objetos se crean a partir de las CLASES, a esta suele llamársela “instanciar un objeto”

Un sistema orientado a objetos es un conjunto de objetos que colaboran entre sí para lograr algo que solos no podrían. Si está bien diseñado hay una relación directa entre los objetos y los conceptos del mundo real que estos representan

Un objeto esta constituido por:

- Estado: Atributo + Valor
- Identidad

La identidad de un objeto esta basada en la ubicacion en memoria de los objetos, que se define en el puntero que poseen, no se basa en un atributo de tipo "ID"

- Comportamiento: metodos funcionando a lo largo del tiempo.

Las reglas de negocio pueden definen en parte el comportamiento de un sistema

El metodo `new()`; lo que hace es crear un nuevo objeto estableciendo sus atributos a travez de los metodos `get()`; y `set()`;

Las clases tienen ciertas características:

- Abstraccion: La abstracción se centra en las características esenciales de un objeto en relación a la perspectiva del observador
- Cohesion: mide la relacion interna de los atributos y metodos de forma que se cumpla con lo establecido en la abstraccion. Utiliza la regla de no repetirse en donde si una clase tiene cohesion, esta clase deberia resolver correctamente una sola tarea y no habria razon de duplicar estos comportamientos o funciones.
- Encapsulamiento: Los atributos permanecen ocultos, los metodos son publicos. El encapsulamiento permite el ocultamiento de información
- Acoplamiento: Hace referencia a la cantidad de relaciones entre las clases

Las clases tambien poseen relaciones:

- Asociacion: Es una relación estructural que especifica que los objetos de un elemento se conectan a los objetos de otro. Permite la visibilidad de una clase a otra, la cual permite que la primera clase conozca los atributos de la otra clase
- Generalización: Es una relación entre un elemento general y un tipo más específico de ese elemento. El hijo puede añadir nueva estructura y comportamiento, o modificar el comportamiento del padre.

Nos permite definir nuevas clases en base a otras clases existentes. El propósito de la herencia es el reuso de comportamiento.

- Polimorfismo:
- Composicion/Agregacion

A la hora de diseñar con el paradigma orientado a objetos, el codigo debe estar:

- Modularizado: es decir debe poseer componenetes independientes, bajo acoplamiento (poca relacion externa con otros elementos) y alta cohesion (mucha relacion interna de un elemento)
- Identificacion de excepciones: se deben tener en cuenta los cursos alternativos ya que si no se tratan, aparecen los errores.
- Tolerancia a fallos: que la existencia de un error no altere el funcionamiento normal del programa

El diseño del software se basa en varios principios:

- Don't repeat yourself: La redundancia provoca inconsistencia y mas trabajo para arreglar un fallo, para evitar estos procesos de depuraciones hay que evitar duplicaciones creando abstracciones.

- Separación de intereses: hay mucha relación con la cohesión de las clases
- Keep it simple: hay que mantener las cosas sencillas utilizando la menor cantidad de abstracciones
- Ley de demeter: Las clases solo deberían poder relacionarse con otra que tienen un vínculo con ellas.
- Tell don't ask: si hace falta algún dato, hay que decirle al objeto que resuelva la lógica de la obtención de esos datos y que los devuelva ya procesados. Cada objeto tiene que usar sus propios datos, estaría mal que una clase solo sirviera para almacenar datos.

Maquina de estados

Un diagrama de transición de estados muestra una máquina de estados, la cual especifica la secuencia de estados en los que un objeto puede estar, los eventos y condiciones que causan que los objetos alcancen esos estados, y las acciones que ocurren cuando esos estados son alcanzados.

El estado es el valor de los atributos en un momento de tiempo. La máquina de estados representa los estados por los que todos los objetos de la clase modelada pueden pasar. La máquina de estado de una clase se construye en función de uno o más atributos de la clase.

Se clasifica en: De comportamiento, dinámico, lógico.

La maquina de estado sirve para descubrir métodos gracias a el análisis del comportamiento del sistema, se la aplica a una clase en donde se consideran todos los estados por los que pueda pasar un objeto de esa clase a lo largo de su vida.

Son usados para modelar el comportamiento dinámico de un elemento de modelado, comúnmente clases cuyos objetos varían el comportamiento en función de su estado. Su uso se basa en explorar el comportamiento de una clase, actor, subsistema o componente y en Modelar sistemas de tiempo real.

La clase a la cual se le aplica la maquina de estados se llama clase de análisis que generalmente son clases transaccionales que contienen:

- Fecha
- Estados
- Cambio de comportamiento en el tiempo

Para su construcción se parte del camino ideal, de un curso normal, y luego se completan las alternativas.

Las entradas que nos pueden servir en mayor o menor medida para hacer la maquina de estados son:

- Glosario

- **Diagrama de clase**
- **Dominio**
- Parte de las fichas de caso de uso
- Descripción de los casos de uso
- **Reglas de negocio**

Y como salida obtenemos la maquina de estados:

- Vista estática (Diagrama de clases)
- Vista dinámica (La maquina de estados en si)

El estado suspendido sirve para dejar de contar el paso de tiempo de un estado. Para que este tipo de estado funcione necesita un pseudo-estado, historia, que le permite volver al anterior estado. Y esta transiciones se describen de la manera retomar(estado).

Los subestados son un estado mas complejo que los demás y que se diagrama como otra maquina de estados. El método inicial es el mismo que con el que entramos al subestado. Cabe recalcar que los métodos de los subestados también deben estar en la clase de análisis.

La forma correcta de modelar una cancelación en un subestado es con un exitpoint, que siempre se dirige a un estado final

Realización de casos de uso

Existe una traza, un rastro que relaciona un caso de uso de requerimiento y el de realización, las instancias de un caso de uso, es cada uno de los cursos de ejecución de estos.

Las realizaciones se diagraman para cada instancia de un caso de uso. Este diagrama tiene tanto una vista estática, el diagrama de clases, como así el diagrama de secuencias.

El objetivo del diagrama es buscar métodos que se necesiten para que el caso de uso funcione correctamente en base a las clases.

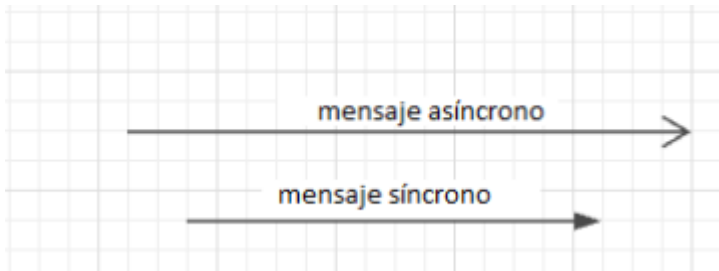
Clases de fabricación, estas se basan en los estereotipos los cuales extienden el lenguaje en base a convenios.

- Clase de Entidad: modela información que podría ser persistente y que puede sobrevivir a una ejecución de un sistema. Cosas, lugares, información necesaria
- Clase límite o frontera (Boundary) modela el comportamiento e información que es dependiente de la frontera del sistema con el ambiente. Modela todo lo que concierne a cualquier vínculo sistema-actor
- Clase de control modela funcionalidad que operar sobre varios objetos diferentes de entidad, haciendo algunos cálculos y retornando el resultado al objeto de boundary. Contiene comportamiento de la lógica de negocio definida en un caso de uso. Tiene

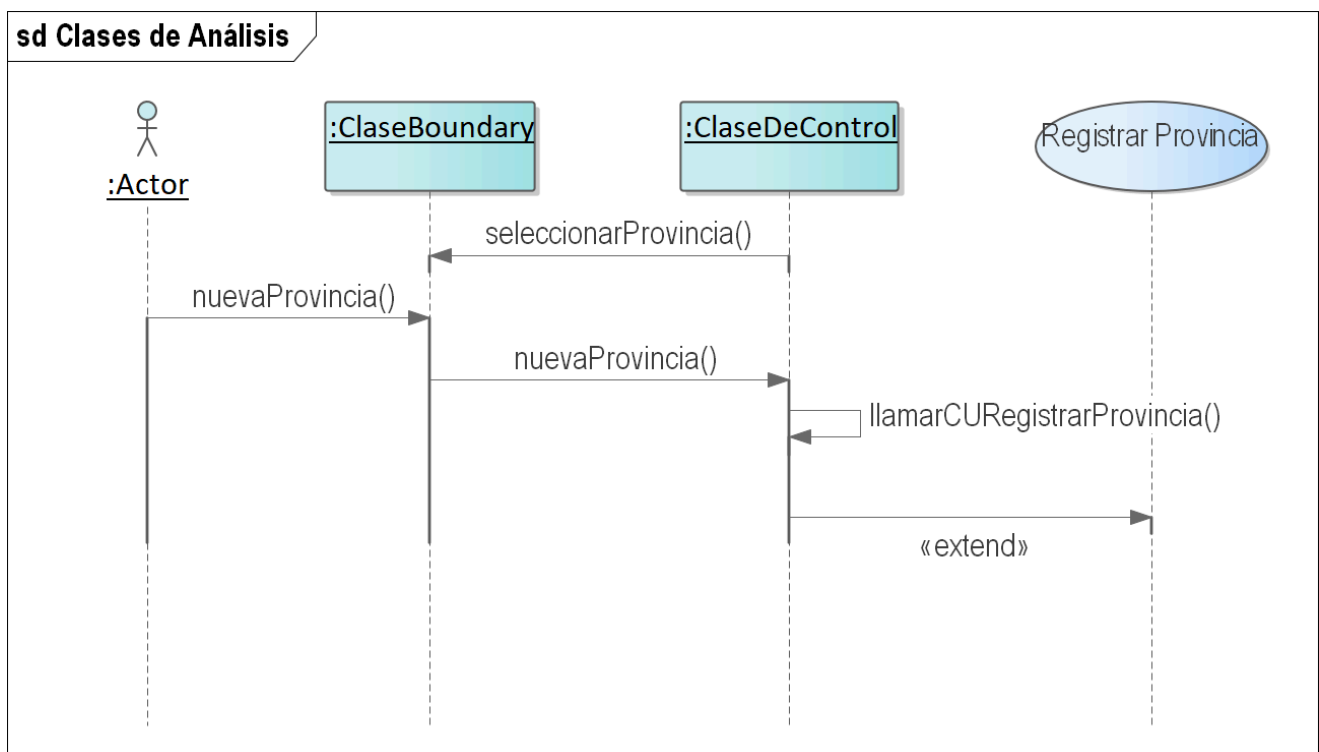
responsabilidades de coordinación de la ejecución de un caso de uso y funciona como intermediario entre las clases de Boundary y las de entidad.

Las clases de fabricación pura son clases inventadas que se usan en el diagrama de secuencia pero no tienen trazas en los casos de uso.

El diagrama es dinámico porque considera el paso del tiempo.



El mensaje síncrono es cuando el objeto emisor queda bloqueado hasta que responde el receptor.



El foco de control se modela para indicar que el objeto tiene el control.

La Visibilidad se relaciona con el hecho de que para que un objeto le envíe un mensaje a otro, debe verlo. Trabajaremos con 2 tipos de visibilidad:

- Visibilidad de atributos: En la estructura se modela con asociación, agregación, composición
- Visibilidad de parámetros: En la estructura se modela con dependencia

Patrones GRASP

Responsabilidades relacionadas con: Hacer - Conocer

Hacer:

- Hacer algo en uno mismo
- Iniciar una acción en otros objetos.
- Controlar y coordinar actividades en otros objetos.

Conocer:

- Estar enterado de los datos privados encapsulados.
- Estar enterado de la existencia de objetos conexos.
- Estar enterado de cosas que se pueden derivar o calcular.

Patrón Controlador

- Problema
¿Quién debería encargarse de atender un evento del sistema?
Un evento del sistema es un evento de alto nivel generado por el actor externo.
- Solución
Asignar la responsabilidad del manejo de un mensaje de los eventos de un sistema, a una clase controladora de esos eventos
- Explicación
Un controlador de caso de uso es una buena alternativa cuando hay muchos eventos del sistema entre varios procesos: asigna su manejo a clases individuales controlables
- Beneficios
Garantiza que los procesos del dominio sean manejados por la capa del dominio y no por la de interfaz. Reflexionar sobre el estado de un caso de uso: asegurar que las operaciones ocurran en una cierta secuencia o saber el estado actual de las operaciones del caso de uso

Patrón Experto

- Problema
¿Cuál es el principio fundamental en virtud del cual se asignan las responsabilidades en el diseño orientado a objetos?
- Solución
Asignar una responsabilidad al experto en información: la clase que cuenta con la información necesaria para cumplir con la responsabilidad.
- Explicación
Los objetos hacen cosas relacionadas con la información que poseen. Puede haber expertos parciales
- Beneficios Bajo el acoplamiento
El comportamiento se distribuye entre las clases que cuentan con la información requerida

Patrón Creador

- Problema
¿Quién debería ser el responsable de crear una nueva instancia de alguna clase?
- Solución
Asignar a la clase B la responsabilidad de crear una instancia de la clase A cuando: B agrega/ contiene los objetos de A; B utiliza específicamente los objetos de A; B tiene los datos de inicialización que serán transmitidos a A cuando sea creado
- Explicación
Encontrar un creador que debemos conectar con el objeto producido, esto soporta el bajo acoplamiento. El agregado “agrega” la parte, el contenedor “contiene” el contenido, el registro “registra”.
- Beneficios
Ayuda al bajo acoplamiento, supone menos dependencias. Como la clase creada tiende a ser visible al creador (razón por la cual se la eligió).

Patrón Bajo Acoplamiento

- Problema
¿Cómo dar soporte a una dependencia escasa y a un aumento de reutilización?
- Solución
Asignar una responsabilidad para mantener bajo el acoplamiento.
- Explicación
Clases más independientes, que reducen el impacto de los cambios, y también más reutilizables. Un grado moderado de acoplamiento es normal para crear un sistema OO, donde los objetos colaboran entre sí.
- Beneficios
No se afectan componentes por cambios de otros componentes. Fáciles de entender por separado. Fáciles de reutilizar.

Patrón Alta Cohesión

- Problema
Las clases con baja cohesión a menudo representan un grado de abstracción alto o han asumido responsabilidades que deberían haber delegado en otros objetos.
- Solución
Asignar una responsabilidad de modo que la cohesión siga siendo alta.
- Explicación
Una clase de alta cohesión tiene un número relativamente pequeño de operaciones con funcionalidad relacionada y poco trabajo por hacer. Colabora con otros objetos para compartir esfuerzo si la tarea es grande.
- Beneficios
Mejoran la claridad y la facilidad de comprensión Simplifican la evolución. A menudo se

genera el bajo acoplamiento. Soporta una mayor capacidad de reutilización, porque una clase muy cohesiva puede destinarse a un propósito muy

Unidad 2 y 3

Workflow de diseño

El diseño es un proceso iterativo mediante el cual se aplican varias técnicas y principios con el objetivo de definir un dispositivo, un proceso o un sistema con suficiente nivel de detalle como para permitir su realización física. Consiste en transformar un modelo lógico en un modelo físico, teniendo en cuenta las restricciones del negocio.

La diferencia entre el análisis y el diseño es que el análisis modela la solución en términos lógicos y el diseño en términos físicos.

Las tendencias de la ingeniería de software son:

- Darle un papel protagonista a la etapa de requerimientos.
- Poner especial atención en la arquitectura del software.
- Utilizar las mismas herramientas para modelar
- Utilizar un proceso

Características del diseño:

- Modelo físico, porque es un plano de la implementación.
- No genérico, específico para una implementación.
- Más formal.
- Dinámico. Muy centrado en la secuencia.
- Manifiesto del diseño del sistema, incluyendo su arquitectura

El diseño a su vez se divide en:

- Diseño Arquitectónico: Da respuesta a los requerimientos de calidad del software, con decisiones estratégicas
- Diseño de Datos: Transforma los requerimientos en las estructuras de datos necesarias para hacer persistente el software, generalmente se usan base de datos transaccionales/relacionales.
- Diseño de Procesos: Transforma elementos estructurales en una descripción procedimental de los componentes del software. Aquí se diseñan los componentes relacionado a la funcional.
- Diseño de Experiencia de Usuario: Es la disciplina relacionada con el diseño, evaluación e implementación de sistemas computacionales interactivos para uso humano.
- Diseño de Formas de Entrada/ Salida: Describe cómo se ingresa información al software y cómo se presentarán las salidas del mismo

- Diseño de Procedimientos Manuales: Describe cómo integra el software al Sistema de Negocio relacionado a los trabajos que se realizan sin software.

El diseño debe cumplir que:

Deberá implementar todos los requisitos explícitos del modelo de análisis, y deberán ajustarse a todos los requisitos implícitos que desea el cliente.

Deberá ser una guía legible y comprensible para aquellos que generan código y para aquellos que comprueban y dan soporte al software.

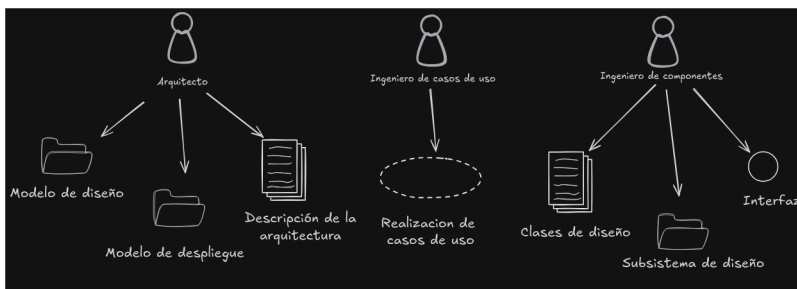
Deberá proporcionar una imagen completa del software que contemple los dominios de comportamiento, funcionales y de datos desde una perspectiva de implementación.

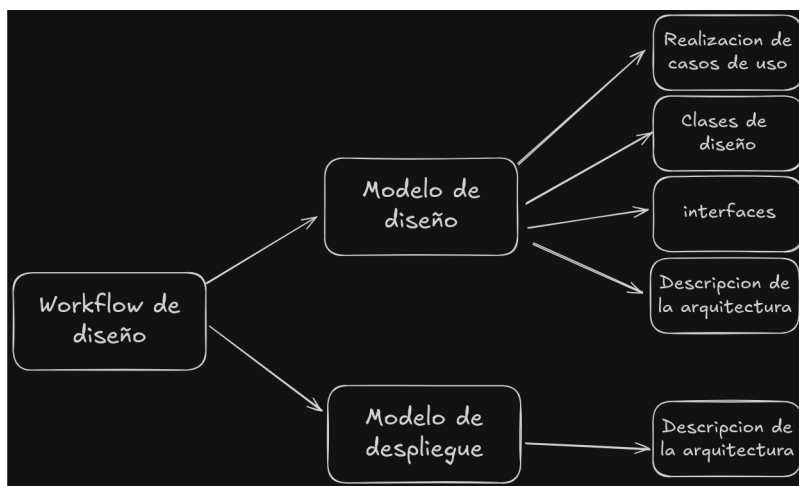
Para evaluar un buen diseño se debe:

- El diseño deberá poderse rastrear hasta el modelo de análisis.
- El diseño no deberá inventar nada que ya esté inventado.
- El diseño deberá «minimizar la distancia intelectual» entre el software y el problema.
- El diseño deberá presentar uniformidad e integración.
- El diseño deberá estructurarse para admitir cambios

Los propósitos del workflow de diseño son:

- Adquirir una comprensión en profundidad de los aspectos relacionados con los requerimientos no funcionales y restricciones tecnológicas.
- Crear una entrada para el workflow de implementación.
- Descomponer los trabajos de implementación en partes manejables y permitiendo la concurrencia entre equipos de desarrollo.
- Capturar interfaces entre subsistemas.
- Crear una abstracción de la solución de diseño sin estar restringidos a una tecnología





Requerimientos no funcionales

Los requerimientos no funcionales deben especificarse de manera objetiva y verificable, cumpliendo estándares.

Estos a su vez se dividen en varios tipos, el único tipo que no tomamos en cuenta es la adecuación funcional que se desarrolla en el análisis.

La Calidad de Producto es relativa a propiedades estáticas del software y propiedades dinámicas de sistemas de computadoras.

Tipos de requerimientos no funcionales:

- Eficiencia de desempeño: relativo a la cantidad de recursos utilizados bajo condiciones establecidas.
 - Comportamiento temporal: el grado con el cual el tiempo de respuesta de un sistema cuando ejecuta sus funciones cumple con los requerimientos.
 - Utilización de recursos: el grado con el cual las cantidades y los tipos de recursos utilizados por un sistema cuando ejecuta sus funciones cumplen con los requerimientos.
 - Capacidad: el grado con el cual los límites máximos de los parámetros de un sistema cumplen con los requerimientos.
- Compatibilidad: El grado con el que un sistema realiza las funciones requeridas compartiendo el mismo entorno de software y hardware.
 - Co-existencia: el grado con el cual un producto puede ejecutar sus funciones requeridas mientras comparte el entorno y los recursos con otros productos sin impactarlos.
 - Interoperatividad: el grado con el cual dos o más sistemas pueden intercambiar información y utilizar esa información.
- Usabilidad: el grado con el cual un sistema puede ser utilizado por usuarios específicos para cumplir con sus objetivos con efectividad, eficiencia y satisfacción en un contexto de uso específico.
 - Reconocimiento de corrección: el grado con el cual los usuarios pueden reconocer que un producto es apropiado para sus necesidades, esto depende de la impresión

inicial y de la documentación asociada.

- Aprendizaje: el grado con el cual un sistema puede ser utilizado por usuarios para cumplir objetivos de aprendizaje establecidos para ejecutar el producto con efectividad, eficiencia, libre de riesgos y satisfacción en un contexto.
- Operabilidad: el grado con el cual un sistema posee atributos que lo hacen fácil de operar y controlar.
- Protección frente a errores del usuario: el grado con el cual un sistema protege los usuarios de cometer errores.
- Estética: el grado con el cual una interfaz de usuario permite una interacción placentera y satisfactoria a los usuarios.
- Accesibilidad: el grado con el cual un producto puede ser utilizado por personas en el sentido amplio de características y capacidades para lograr un objetivo establecido en un contexto de uso
- Fiabilidad: El grado con el que un sistema ejecuta funciones bajo condiciones por un período de tiempo determinado.
 - Madurez: el grado con el cual un sistema cumple las necesidades de confiabilidad durante la operación normal. La Detección de fallas se soluciona mediante Monitoreo.
 - Disponibilidad: el grado con el cual un sistema está operativo y accesible cuando es requerido su uso. Externamente, la disponibilidad puede definirse como el porcentaje de tiempo durante el cual el sistema está arriba. También se define como una combinación de madurez, la tolerancia a fallas y la capacidad de recuperación.
 - Tolerancia a fallas: grado el que un sistema opera según lo previsto a pesar de fallas en el software o el hardware.
 - Capacidad de recuperación: el grado con el cual en un evento de interrupción o falla, un sistema puede restaurar la información afectada y reestablecer el estado deseado del sistema. Esto se soluciona con Respaldo, sincronización y rollback.
- Seguridad: El grado en que un sistema protege información y datos de modo que personas u otro sistemas tengan el nivel de acceso a datos apropiado a su tipo y nivel de autorización
 - Confidencialidad: el grado con el que un sistema asegura que los datos son accesibles sólo a aquellos autorizados a acceder.
 - Integridad: el grado con el que un sistema previene el acceso no autorizado o modificaciones a un programa de computadoras o datos.
 - No repudio: el grado con el que acciones pueden ser probados que existieron de manera que esas acciones no puedan ser luego repudiados.
 - Autenticidad: el grado con el cuál pueda establecerse la identidad de una persona o un recurso pueda ser demostrada
 - Responsabilidad: el grado con el cuál una acción de una entidad puede trazarse unívocamente hacia esa entidad

- **Mantenibilidad:** Se relaciona con la capacidad de facilitar las actividades de mantenimiento.
 - **Modularidad:** el grado con el cual un sistema está compuesto por componentes discretos de manera que el cambio en uno de sus componentes tiene impacto mínimo en el resto.
 - **Reusabilidad:** el grado con el cual un activo puede ser utilizado en más de un sistema o en la construcción de otros activos.
 - **Analizabilidad:** el grado de eficiencia con el que es posible evaluar el impacto de un producto que requiere ser modificado o diagnosticarlo a cerca de sus fallas y deficiencias o identificar las partes que necesitan ser modificadas.
 - **Capacidad de ser modificado:** el grado en el que un sistema puede ser eficiente y eficazmente modificado sin introducir defectos o degradar la calidad actual del producto.
 - **Capacidad de ser probado:** el grado de eficacia y eficiencia con que los criterios de prueba pueden establecerse para un sistema y las pruebas pueden ejecutarse para establecer si los criterios se cumplen.
- **Portabilidad:** el grado de eficacia y eficiencia con el que un sistema puede ser transferido de un hardware, software u otro entorno operativo a otro.
 - **Adaptabilidad:** el grado con el cual un sistema puede ser efectiva y eficientemente adaptado para otros hardware, software o entornos operacionales o de uso diferentes o evolucionados.
 - **Facilidad de instalación:** el grado de efectividad y eficiencia con el que un sistema puede ser instalado o desinstalado exitosamente en determinado entorno.
 - **Capacidad de ser reemplazado:** el grado con el cual un producto puede reemplazar otro producto de software específico con el mismo propósito y en el mismo entorno.

Arquitectura de software

Arquitecto de software (rol)

Es el encargado de:

- Revisa y negocia los requerimientos
- Documentar la Arquitectura
- Comunica la arquitectura, asegurándose que los involucrados comprendan.
- Direcciona requerimientos no funcionales a la arquitectura
- Configura la arquitectura de hardware
- Se asegura que la arquitectura es respetada
- Trabaja con el Administrador del Proyecto, ayudando en la planificación, la estimación la distribución de tareas y la calendarización del proyecto.

Las ventajas de que el arquitecto realice la documentación de la arquitectura, la cual establece las decisiones que se toman para desarrollar la arquitectura misma, se basan en

implementar una Comunicación con los involucrados en el desarrollo, se crea un mejor Análisis del sistema y permite la Reutilización a gran escala

Proceso de arquitectura

El contexto de la arquitectura se estructura del diseño de la arquitectura tanto del hardware como el software, un diseño detallado de la codificación, integración y testing y un análisis del dominio, requerimientos y análisis de riesgo.

El diseño arquitectónico es la asignación de modelos de requerimientos esenciales a una tecnología específica, es el plan del Diseño.

El proceso de diseño propiamente se estructura por 3 pasos:

1. Determinar los requerimientos arquitectónicos: Implica la creación de una definición o modelo de los requerimientos que conducirán el diseño arquitectónico y su priorización
2. Diseño Arquitectónico: Implica la definición de la estructura y las responsabilidades de los componentes que constituirán la arquitectura
3. Validación: Implica “probar” la arquitectura, comúnmente recorriendo el diseño, contra los requerimientos existentes y cualquier requerimiento futuro, posible o conocido

La elección de que modelo arquitectónico se usa, se hace en base a los requerimientos no funcionales, específicamente en:

- Desempeño (Performance): Si es un requerimiento crítico, esto sugiere que la arquitectura se debe diseñar para localizar operaciones críticas dentro de un número reducido de subsistemas con poca comunicación (hasta donde sea posible) entre estos subsistemas, es decir componentes de granularidad gruesa para reducir componentes de comunicación.
- Seguridad: Si es un requerimiento crítico, esto sugiere utilizar una estructura en capas para la arquitectura con los recursos más críticos protegidos con capas internas y un alto nivel de validación aplicado a esas capas.
- Protección: Si es un requerimiento crítico, sugiere que la arquitectura debe diseñarse para incluir operaciones relacionadas con la protección se localicen en un solo subsistema o en un número reducido de subsistemas. Esto reduce costos y problemas de validación.
- Disponibilidad: Si es un requerimiento crítico, debe diseñarse una arquitectura que incluya componentes redundantes de tal forma que puedan reemplazarse y actualizarse sin detener el sistema.
- Mantenibilidad: Si es un requerimiento crítico, la arquitectura debe diseñarse utilizando componentes autocontenidos de granularidad fina que puedan cambiarse con facilidad. Los productores de datos separados de los consumidores y las estructuras de datos compartidas deben evitarse

Para concluir se puede decir que:

- Utilizar componentes de granularidad alta mejora la performance, pero reduce mantenibilidad.
- La introducción de datos redundantes mejora la disponibilidad pero hace la seguridad más difícil.
- La localización de aspectos de seguridad relacionados usualmente significa más comunicación por lo tanto degrada la performance.

Los conflictos arquitectónicos ocurren cuando se realiza la priorización de los requerimientos de arquitectura ya que la priorización es engañosa dado que los requerimientos entran en conflicto entre sí

Los requerimientos significativos para la arquitectura se definen como aquellos que si no los establecemos en la etapa de desarrollo se van a realizar de una forma distinta y eso va a generar costos.

Inicialmente es suficiente ubicar los requerimientos en tres categorías

- Alto: la aplicación debe soportar este requerimiento. Conducen el diseño de arquitectura.
- Medio: necesitará ser soportado en alguna etapa, pero no necesariamente en el primer release.
- Bajo: estos son parte de la lista de deseos. Las soluciones que los incluyen son deseables, pero no son conductores del diseño.

En el proceso del diseño arquitectónico se pueden identificar tanto las entradas como las salidas de este proceso.

Entradas:

Framework de arquitectura: El framework define comunicación general para los componentes y se seleccionan en base a:

- La mayoría de las aplicaciones responden a un pequeño número de arquitecturas probadas y bien comprendidas.
- Utilizar soluciones conocidas minimiza los riesgos.
- No hay una fórmula mágica para diseñar un framework de arquitectura.
- Un pre-requisito: conocer los estilos arquitectónicos principales para abordar ciertos atributos de calidad.

Distribución de componentes: Se definen los componentes principales que comprenderán el diseño.

Se deben identificar:

- Componentes principales de la aplicación y como se insertan en el framework.
- Servicios o interfaces que cada componente soporta.

- Responsabilidades del Componente, definiendo que será invocado para hacer cuando reciba un pedido.
- Dependencias entre componentes.
- Particiones en la arquitectura que son candidatas para distribución entre servidores de la red.

A la hora de diseñar los componentes, hay que tener en cuenta:

- Minimizar las dependencias entre componentes.
- Diseñar componentes que encapsulen un conjunto de responsabilidades altamente cohesivas. Esto limita los tipos de cambios y reduce esfuerzo de mantenimiento y testing.
- Aislar dependencias entre el middleware y cualquier COT de infraestructura tecnológica. Es más difícil de construir, pero introduce penalidades de performance.
- Utilizar la descomposición para estructurar componentes jerárquicamente.
- Minimizar las llamadas entre componentes

Salidas:

Un modelo es una descripción completa de un sistema desde una perspectiva particular y a un nivel de abstracción específico.

Una vista es una proyección de un modelo (o modelos), el cual es visto de una perspectiva o punto de ventaja particular , que omite entidades que no son relevantes para esta perspectiva.

Un punto de vista es una definición (o descripción) de una vista Contenido, significado y representación (lenguajes, técnicas de modelado)

El modelo de vistas que se usa es el que esta estructurado de:

- Vista de Diseño la cual esta compuesta de clases, interfaces, colaboraciones en donde se define el vocabulario y funcionalidad
- Vista de Implementación que esta compuesta de componentes la cual define el montaje del sistema y la administración de la configuración
- Vista de Proceso que esta compuesta de Clases Activas y que define performance y escalabilidad
- Vista de Despliegue compuesta por nodos que definen la topología, distribución Entrega e instalación.

No todos los sistemas requieren todas las vistas, si tiene Procesador Único se saltea la vista de despliegue, si son Procesos Simples se saltea la vista de proceso, si son Programas muy pequeños se saltea la vista de implementación.

La ultima fase del proceso de diseño es la validacion, esta etapa consiste en que:

- Ayuda a incrementar la confianza del equipo de diseño en que la arquitectura cumpla con su propósito.
- Debe realizarse con las restricciones de tiempo y presupuesto del proyecto.
- El truco es ser tan riguroso y eficiente como sea posible.
- Es un desafío validar la arquitectura ya que finalmente es un diseño y no puede ejecutarse ni probarse para ver si cumple los requerimientos.

Existen dos técnicas principales que tienen un uso probado. Ambas ayudan a identificar fallas y debilidades, áreas de riesgo potencial para las posteriores actividades de construcción.

1. Probar Manualmente la Arquitectura utilizando escenarios.

Se trabaja la respuesta de la arquitectura a esos estímulos destacando las implicancias de la arquitectura en las decisiones de diseño.

Los escenarios son artefactos simples que implican la definición de estímulos que tendrán impacto en la arquitectura.

Los escenarios están relacionados con aspectos arquitectónicos y estos ayudan a destacar consecuencias de las decisiones arquitectónicas que están encapsuladas en el diseño.

2. Construir prototipos que crea un arquetipo de la aplicación deseada.

Los escenarios son una técnica útil para validar la arquitectura, sin embargo, algunos escenarios no son tan simples.

Los prototipos son versiones mínimas, restringidas de la aplicación, creadas específicamente para probar algún aspecto riesgoso o pobremente comprendido del diseño.

Los prototipos son utilizados comúnmente para dos propósitos.

- Prueba de Conceptos: ¿Puede la arquitectura como fue diseñada ser construida de manera tal que satisfaga los requerimientos?
- Prueba de Tecnología: ¿La tecnología elegida (middleware, aplicaciones integradas, librerías, etc.) para implementar la arquitectura se comporta como es esperado?

En ambos casos proveen una evidencia que de otra forma no se puede validar.

Productos (arquitectura)

Sistemas distribuidos

En términos computacionales hay 3 tipos de sistemas:

- Sistemas Personales: no son distribuidos y han sido diseñados para ejecutarse en una computadora personal o estación de trabajo.

- Sistemas Embebidos: se ejecutan en un único procesador o en un grupo integrado de procesadores.
- Sistemas Distribuidos: donde el sistema de software se ejecuta en un grupo de procesadores cooperativos integrado conectados por una red

Virtualmente todos los sistemas grandes basados en computadoras son ahora sistemas distribuidos ya que el procesamiento de información está distribuido en varios procesadores.

Características:

- Compartición de Recursos: recursos de hardware y software asociados por una red.
- Apertura: son abiertos, se diseñan sobre protocolos estándar que combinan equipamiento y software de diferentes vendedores.
- Concurrencia: varios procesos operando al mismo tiempo sobre diferentes computadoras de la red.
- Tolerancia a Fallas: la disponibilidad de varios recursos permite esta capacidad.
- Escalabilidad: los sistemas pueden crecer incrementando recursos para cubrir demandas nuevas. Tres dimensiones de la escalabilidad:
 - Tamaño: debe ser posible agregar más recursos.
 - Distribución: dispersar los recursos sin reducir su rendimiento.
 - Manejabilidad: debe poder administrarse un sistema conforme aumenta su tamaño, incluso si partes del sistema se ubican en organizaciones diferentes

Desventajas:

- Complejidad: los hace más difíciles de comprender y probar.
- Seguridad: dificulta asegurar la integridad y la degradación del servicio.
 - Intercepción, Interrupción, Modificación, Fabricación
- Manejabilidad: Los defectos pueden propagarse de una máquina a otra, esto implica que es más difícil de gestionar y administrar.
- Impredecibilidad: la respuesta depende de la carga total en el sistema, de la organización y de la red.

Falacias:

- La red no esta disponible
- La latencia es 0
- El ancho de banda es infinito
- La red es segura
- La topologia nunca cambia
- Hay un único administrador
- El costo de transporte es 0
- La red es homogénea

Arquitectura monolítica

Son aplicaciones autosuficientes que contienen toda la funcionalidad necesaria para realizar la tarea para la cual fueron diseñadas, sin contar con dependencias externas que la complementen. La capacidad más notable que tiene es la Independencia

Permite crear aplicaciones en las que todos los componentes y módulos se empaquetan y se implementan como uno sólo. Todos los módulos y niveles están unidos entre sí, actuando como un cuerpo singular.

Características:

- En un servicio monolítico, la mayor parte del código del lado del servidor está en un programa que se comunica con una o más bases de datos.
- Dentro de un servidor monolítico, el código aún podría dividirse en componentes y separarse en módulos individuales.
- Las únicas API definidas suelen ser:
 - A) Entre la interfaz de usuario y el servidor (en cualquier protocolo REST/HTTP)
 - B) Entre el servidor y las bases de datos (en cualquier lenguaje de consulta)
 - C) Entre el servidor y sus dependencias externas.

Ventajas:

- Fácil de desarrollar, especialmente desde cero.
- Fácil de probar
- Es simple escalar, utilizando un balanceador de cargas.
- Pocos puntos de fallo, al no depender de nada más.
- Autónoma
- Suelen ser más rápidas, procesamiento local y no necesita consumir procesos distribuidos.

Desventajas:

- Crea una aplicación más compleja.
- Si falla, falla todo.
- Difíciles de mantener y depurar.
- El cambio en una característica o una parte de la aplicación requiere que la aplicación completa se despliegue nuevamente.
- Un aumento en el tamaño de la aplicación aumenta la complejidad, lo que dificulta su escalado después de un tiempo.
- La escalabilidad en un módulo requiere escalar toda la aplicación.
- Anclado a un Stack tecnológico

Spaguetti Architecture

- Es el estilo arquitectónico que tiene un producto cuando no se define una arquitectura.
- Surge de un equipo que no tiene un plan sobre cómo se va a construir su producto.
- Las partes del sistema a menudo están altamente acopladas, el mantenimiento es muy difícil.
- Explicar cómo encaja todo a alguien nuevo es casi imposible

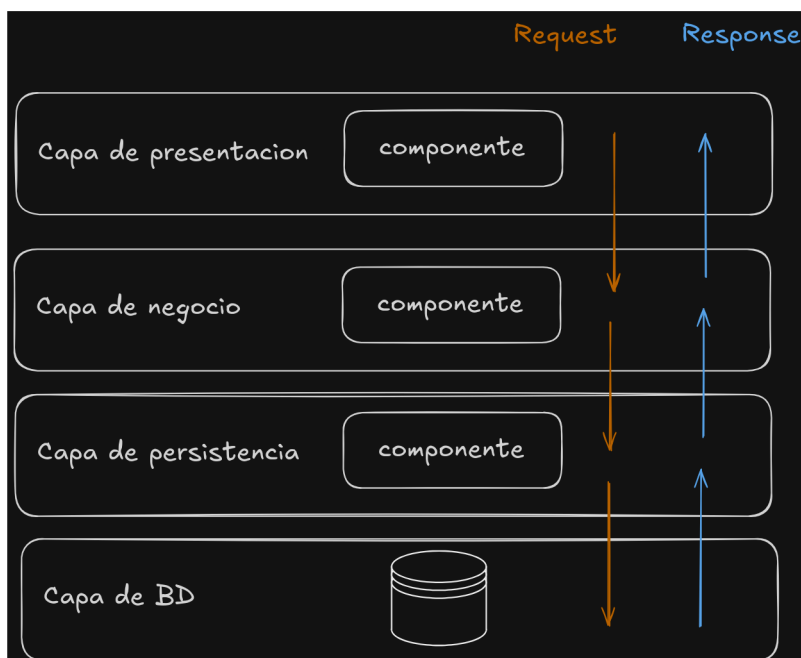
Arquitectura Estratificada o en capas (estilo)

Es tal vez el más común, tanto que muchos desarrolladores asumen que todos los sistemas deberían ser estratificados. Se encuentran sistemas forzados a ser estratificados.

Organiza los componentes de una aplicación en una serie de capas, cada una con una responsabilidad específica. Aplicando el concepto de separación de intereses

Las capas más comunes son:

- Capa de presentación: responsable de manejar la entrada y salida del usuario y de mostrar la interfaz de usuario.
- Capa de Lógica de Negocio:
 - Capa de aplicación: contiene la lógica de negocio de la aplicación, incluidas las reglas y procesos que gobiernan el comportamiento del sistema.
 - Capa de dominio: representa los datos y los conceptos de negocio de la aplicación, y es responsable de gestionar los datos y las reglas de negocio.
- Capa de acceso a datos: responsable de acceder y manipular datos, generalmente a través de una base de datos.
- Capa de infraestructura: incluye los sistemas y tecnologías subyacentes que respaldan la aplicación, como el sistema operativo, el sistema de gestión de bases de datos y los protocolos de red.



Ventajas:

- Fácil de desarrollar: fácil de implementar, es muy conocido y una gran mayoría de las aplicaciones la utilizan.
- Fácil de probar: permite probar por separada cada capa.
- Fácil de mantener: fácil detectar el origen de un defecto para corregirlo, o identificar donde se debe aplicar un cambio.
- Seguridad: la separación de capas permite el aislamiento de los servidores en subredes diferentes, lo que hace más difícil realizar ataques.

Desventajas:

- Performance: La comunicación por la red o internet es una de las tareas más lentas de un sistema, incluso, más que el mismo procesamiento de los datos, por lo que el hecho de tener que comunicarnos de capa en capa genera un degrado significativo en el performance.
- Escalabilidad: al ser monolíticas, son difíciles de escalar, aunque es posible replicar la aplicación completa en varios nodos, lo que provoca un escalado monolítico.
- Complejidad de despliegue: es necesario desplegar los componentes de abajo arriba, lo que crea una dependencia en el despliegue. Un pequeño cambio puede requerir el despliegue completo de la aplicación.
- Anclado a un Stack tecnológico
- Tolerancia a los fallos: Si una capa falla, todas las capas superiores comienzan a fallar en cascada

Estilo Partición de Dominio

Separan los componentes de nivel superior por flujos de trabajo y/o dominios, se hacen capas en función del funcionamiento del negocio.

Ventajas:

- Modelado más de cerca de cómo funciona el negocio en lugar de hacia detalles de implementación.
- Se alinea más estrechamente con el monolítico modular y la arquitectura de microservicios.
- El flujo de mensajes coincide con el dominio del problema
- Fácil de migrar datos y componentes a la arquitectura distribuida

Desventaja:

- El código de personalización aparece en varios lugares

Arquitectura hexagonal

La arquitectura está simbolizada por un hexágono, que proporciona un amplio espacio para representar múltiples puertos y adaptadores

La lógica de negocio crea puertos que se pueden conectar a varios adaptadores de forma intercambiable.

Las interfaces creadas por el código de negocio se denominan puertos, que se consideran internos tal como los define el negocio.

Los adaptadores representan código externo que cierra la brecha entre el puerto y el resto del código del lado del usuario o del servidor.

Permite que la entrada de usuarios o sistemas externos llegue en un Puerto a través de un Adaptador, y permite que la salida se envíe desde la Aplicación a través de un Puerto a un Adaptador. Esto crea una capa de abstracción que protege el núcleo de una aplicación y la aísla de herramientas y tecnologías externas

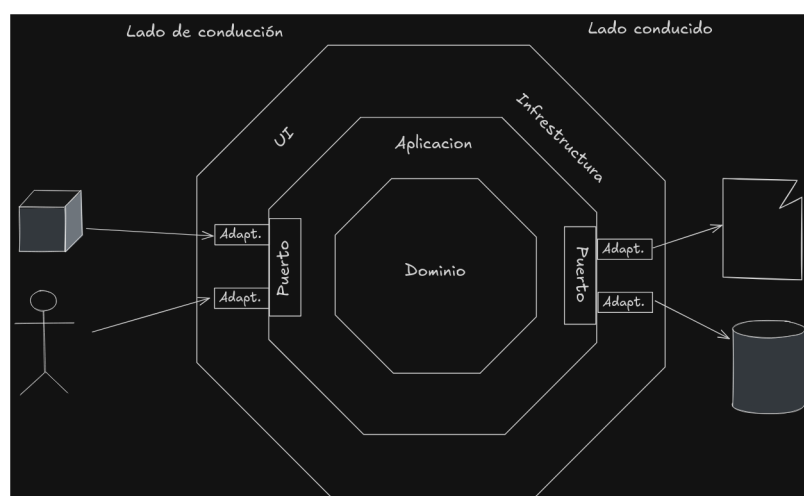
Puertos: punto de entrada independiente de la tecnología, determina la interfaz que permitirá a los actores externos comunicarse con la Aplicación, independientemente de quién o qué implementará dicha interfaz.

Los puertos también permiten que la Aplicación se comunique con sistemas o servicios externos, como bases de datos, intermediarios de mensajes, otras aplicaciones, etc.

Adaptadores: iniciará la interacción con la Aplicación a través de un Puerto, utilizando una tecnología específica, por ejemplo, un controlador REST representaría un adaptador que permite a un cliente comunicarse con la Aplicación.

Pueden existir tantos Adaptadores para un mismo Puerto como sean necesarios sin que esto represente un riesgo para los Puertos o la propia Aplicación.

Aplicación: representada por un hexágono que recibe comandos o consultas en los Puertos y envía solicitudes a otros actores externos, como bases de datos, también a través de los Puertos.



Los actores conductores (o primarios) son los que inician la interacción y siempre están representados en el lado izquierdo. Un adaptador de conducción podría ser un controlador que es el que toma la entrada y la pasa a la Aplicación a través de un Puerto.

Los actores conducidos (o secundarios) son aquellos a los que la Aplicación “impulsa a comportarse”. La aplicación llama a un adaptador de base de datos para que obtenga un determinado conjunto de datos de la persistencia.

Los puertos estarán representados como interfaces en el código.

Los Adaptadores de conducción utilizarán un Puerto y un Servicio de Aplicación implementará la Interfaz definida por el Puerto, en este caso tanto la interfaz como la implementación del Puerto están dentro del Hexágono.

Los adaptadores conducidos implementarán el Puerto y un Servicio de Aplicación lo utilizará; en este caso el Puerto está dentro del Hexágono, pero la implementación está en el Adaptador, por lo tanto, fuera del Hexágono.

En el lado de conducción, el Adaptador depende del Puerto, que es implementado por el Servicio de Aplicación, por lo tanto, el Adaptador no sabe quién reaccionará a sus invocaciones, sólo sabe qué métodos están garantizados que estarán disponibles, por lo tanto, depende de una abstracción.

En el lado Conducido, el Servicio de Aplicación es el que depende del Puerto, y el Adaptador es el que implementa la Interfaz del Puerto, invirtiendo efectivamente la dependencia ya que el adaptador de 'bajo nivel' (es decir, el repositorio de la base de datos) se ve obligado a implementar la abstracción definida en el núcleo de la aplicación, que es de "nivel superior".

La "regla de dependencia" proporciona una solución al garantizar que todas las dependencias del código fuente apunten hacia adentro, hacia el núcleo de la aplicación.

El "principio de inversión de dependencia": El puerto todavía está definido por una interfaz, pero las relaciones entre clases se invierten. Esto permite controlar la dirección de una dependencia del código; para puertos secundarios y adaptadores, es opuesta a la dirección de llamada

Pasos para aplicar arquitectura hexagonal:

- Identificar la lógica de su dominio principal y encapsúlela en una o más clases o módulos. Este es el núcleo de tu aplicación.
- Identificar sus actores externos y sus interacciones con su sistema. Estos son sus puertos primario y secundario
- Definir interfaces abstractas para cada puerto que especifiquen qué métodos u operaciones están disponibles para la comunicación.
- Implementar adaptadores concretos para cada puerto que manejen la comunicación con los actores externos.
- Conecte sus adaptadores a su núcleo mediante inyección de dependencia o inversión de control. De esta manera, puedes cambiar o reemplazar fácilmente tus adaptadores sin afectar tu núcleo

Beneficios:

- Acoplamiento bajo al desacoplar el núcleo de los sistemas externos y sus detalles técnicos.
- Alta cohesión al manejar las inquietudes técnicas con los adaptadores y mantener el enfoque en la lógica de negocio y de aplicaciones dentro del núcleo.
- Capacidad de prueba mejorada.
- Mantenimiento más fácil.
- Mejor modelado de dominio.
- Escalabilidad.
- Intercambiabilidad cambiando adaptadores sin cambiar el núcleo.
- Reusabilidad mediante el uso de diferentes adaptadores.
- Legibilidad mejorada

Cliente servidor

Definicion

Muchos estilos de arquitectura se ocupan de cómo separar eficientemente las partes del sistema. Un estilo fundamental en la arquitectura separa la funcionalidad técnica entre frontend y backend, denominada arquitectura de dos niveles o cliente/servidor.

Formas destacadas de implementar el patrón C/S:

- El patrón cliente-servidor de acceso a datos remoto
- El patrón cliente-servidor de presentación remota
- El patrón cliente-servidor de datos de lógica dividida

Ventajas:

- Centralización: lo que impide que los clientes conserven información desactualizada.
- Seguridad: El servidor por lo general está protegido por firewall o subredes que impiden acceder a la base de datos o los recursos sin pasar por el servidor.
- Fácil de instalar (cliente): El cliente es por lo general una aplicación simple que no tiene dependencias, por lo que es muy fácil de instalar.
- Separación de responsabilidades: La arquitectura cliente-servidor permite implementar la lógica de negocio de forma separada del cliente.
- Portabilidad: Una de las ventajas de tener dos aplicaciones es que podemos desarrollar cada parte para correr en diferentes plataformas, por ejemplo, el servidor solo en Linux, mientras que el cliente podría ser multiplataforma

Desventajas:

- Actualizaciones (clientes pesados): Una de las complicaciones es gestionar las actualizaciones en los clientes, pues puede haber muchos terminales con el cliente

instalado y tenemos que asegurar que todas sean actualizadas cuando salga una nueva versión.

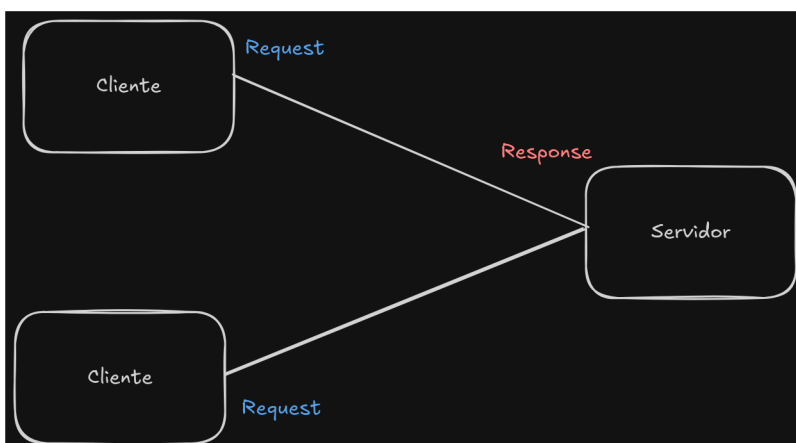
- Concurrencia: Una cantidad no esperada de usuarios concurrentes puede ser un problema para el servidor, quien tendrá que atender todas las peticiones de forma simultánea, aunque se puede mitigar con una estrategia de escalamiento, siempre será un problema que tendremos que tener presente.
- Todo o nada: Si el servidor se cae, todos los clientes quedarán totalmente inoperables.
- Protocolos de bajo nivel: Los protocolos más utilizados para establecer comunicación entre el cliente y el servidor suelen ser de bajo nivel, como Sockets, HTTP, RPC, etc. Lo que puede implicar un reto para los desarrolladores.
- Depuración: Es difícil analizar un error, pues los clientes están distribuidos en diferentes máquinas, incluso, equipos a los cuales no tenemos acceso, lo que hace complicado recopilar la traza del error

2 capas

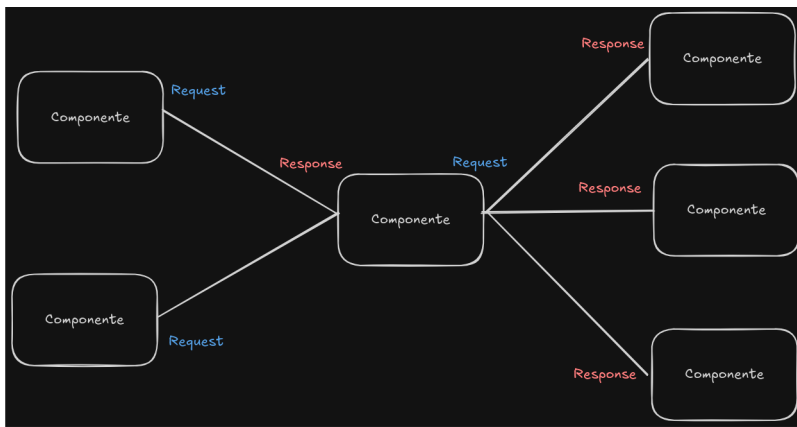
La aplicación es modelada como un conjunto de servicios que son provistos por servidores y un conjunto de clientes que usan esos servicios.

Los clientes conocen a los servidores y deben saber cómo buscarlos, pero los servidores no necesitan conocer a los clientes.

Clientes y Servidores son procesos lógicos. Los clientes inician la comunicación. Los servidores no conocen la identidad de los clientes hasta que se han contactado.



N-capas



Propiedades:

- Separación de Intereses: capas diferentes y claramente divididos para presentación, negocios y manejo de datos.
- Comunicaciones sincrónicas: la comunicación entre niveles es pedido- respuesta sincrónica. Los pedidos emanan en una única dirección del nivel del cliente a los niveles web y de lógica de negocio al nivel EIS. Cada nivel espera la respuesta del otro nivel antes de proseguir.
- Distribución flexible: no hay restricciones para la distribución multi- capas de la aplicación. Todas las capas pueden correr en la misma máquina o en su propia máquina. En aplicaciones WEB, el cliente usualmente corre en un browser de una PC de escritorio de un usuario, comunicándose remotamente con Internet con componentes a nivel web

Maestro - esclavo

Peer-to-peer

Modelo - vista - controlador (MVC)

Basado en tres tipos de objetos:

- Modelo: es responsable de la lógica empresarial de la aplicación. Gestiona el estado de la aplicación. Esto también incluye la lectura y escritura de datos, el estado persistente de la aplicación e incluso puede incluir tareas relacionadas con la gestión de datos, como la red y la validación de datos.
- Vistas: objeto presentación en pantalla, este componente tiene dos tareas importantes: presentar datos al usuario y manejar la interacción del usuario.
- Controlador: define la forma en que la interfaz reacciona a la entrada del usuario. Lo hace para incrementar flexibilidad y reutilización

Características:

- Desacopla el modelo de las vistas estableciendo un modelo de suscripción/ notificación.
- La vista debe asegurarse que su apariencia refleja el estado del modelo.
- Frente a cambios el modelo se encarga de avisar a las vistas que dependen de él.

- La vista se actualiza a sí misma.
- Se pueden crear varias vistas de un modelo, para ofrecer diferentes presentaciones.

Separa presentación e interacción de los datos del sistema.

Se usa cuando existen múltiples formas de ver e interactuar con los datos. También se utiliza al desconocerse los requerimientos futuros para la interacción y presentación.

Ventajas

- Permite que los datos cambien de manera independiente de su representación y viceversa.
- Soporte a distintas representaciones de los mismos datos, y los cambios en una representación se muestran en todos ellos.

Desventajas

- Puede implicar código adicional y complejidad de código cuando el modelo de datos y las interacciones son simples.

Sus metas son aumentar.

- Modularidad
- Flexibilidad
- Testeabilidad
- Mantenibilidad

Arquitecturas Orientadas a Servicio (SOA)

Las arquitecturas orientadas a servicios (SOA) dividen los programas monolíticos en servicios para cada función independiente dividida en componentes. Permiten desarrollar y razonar diferentes servicios de forma independiente (y en paralelo), si es necesario.

Las aplicaciones están diseñadas para ser integradas con otras y deben de exponer como servicios todas las operaciones necesarias para que otras aplicaciones puedan interactuar.

- La interfaz entre cada uno de estos servicios es una capa API definida formalmente.
- Los servicios están débilmente acoplados, lo que significa que un servicio totalmente nuevo puede reutilizar los otros servicios.
- Los servicios deben construirse con estándares abiertos, es decir que es independiente del sistema operativo o del lenguaje de programación.

Web services

Un Web Service, debe de publicar un WSDL (Web Services Description Language), el cual es un contrato que describe las operaciones que expone el servicio:

- Protocolos utilizados: principalmente HTTP, también puede ser FTP o SMTP; dado que SOA no define los protocolos.
- El mensaje (XML) debe cumplir con el contrato (WSDL).

- Luego, el mensaje es envuelto en el protocolo SOAP (Simple Object Access Protocol), el cual es otro protocolo estándar que divide el mensaje en dos secciones, el header y el payload

Composición de servicios

SOA hace foco en la flexibilidad por sobre la optimización. No es especialmente rápida, dada la latencia en el consumo de servicios (por lo general distribuidos).

La flexibilidad permite la construcción de nuevos servicios utilizando la infraestructura de servicios existentes.

El nivel de composición va subiendo la granularidad, puede que el primer nivel sean servicios básicos de CRUD (Altas, Bajas, Cambios, Borrado) y los servicios de más nivel son operaciones completas de transacciones

Con service bus

El service bus permite crear abstracciones de los servicios expuestos para evitar acoplamiento. Evita las conexiones punto a punto entre servicios. Se puede agregar lógica adicional para: seguridad, auditoría, medir invocaciones, tiempos de respuesta, notificar errores.

Existentes: Oracle SB, IBM Message Broker, Mulesoft, Jboss ESB

Desventajas:

- Performance
- Volumetría: para procesar mucha información es recomendable utilizar otras arquitecturas.
- Gobierno: es necesario crear un Gobierno que ponga orden sobre el uso y la creación de nuevos servicios.
- Más puntos de falla: es necesario implementar estrategias de falla.

Ventajas:

- Reduce el acoplamiento: las referencias son un servicio que puede estar en cualquier parte.
- Testabilidad: fáciles de probar, ya que es posible probar de forma individual cada servicio.
- Reutilización: Los servicios SOA son desarrollados para hacer una sola tarea.
- Agilidad: Permite crear nuevos servicios a partir de otros existentes.
- Escalabilidad: se puede agregar varias instancias de un servicio y balancear la carga desde el ESB.
- Interoperable: permite que aplicaciones heterogéneas se comuniquen de forma homogénea mediante el uso de estándares abiertos.

Event Driven Architecture (EDA)

Arquitectura asíncrona y distribuida, pensada para crear aplicaciones altamente escalables.

En una arquitectura EDA se espera que las aplicaciones lancen diversos “eventos” para que otros componentes puedan reaccionar a ellos, procesarlos y posiblemente generar nuevos eventos para que otros componentes continúen con el trabajo.

Un evento lo podemos definir como un cambio significativo en el estado de la aplicación, ya sea por un dato que cambio o alguna acción concreta en el sistema que merece la pena ser observada para tomar acciones en consecuencia.

Un sistema controlado por eventos "reacciona" a los eventos que ocurren en una interfaz de usuario o internamente. Se basa en procesadores de eventos. El flujo de eventos a través del sistema puede tener lugar de dos maneras diferentes. (Broker o Mediator)

- Bajo la topología de Broker, no existe un control centralizado del procesamiento de eventos.
- Bajo la topología del Mediator hay algún control central.

Para completar una tarea se debe colocar el Evento por parte del productor, esperar a que el consumidor lo tome, y generar nuevos Eventos

Ventajas:

- Escalabilidad: permite que cada consumidor (Event Consumer) puede escalar de forma independiente y reduce al máximo el acoplamiento entre los componentes.
- Despliegue: Debido al bajo acoplamiento entre los componentes, es posible el despliegue sin preocuparse por dependencias o precondiciones, al final, los componentes solamente se suscriben para recibir eventos y reaccionar ante ellos.
- Performance: EDA necesita pasar por una serie de pasos para completar una tarea, agregando retrasos en cada paso sin embargo, la naturaleza asíncrona de EDA hace que esta desventaja se supere mediante el procesamiento en paralelo.
- Flexibilidad: EDA permite responder rápidamente a un entorno cambiante, debido a que cada componente procesador de eventos tiene una sola responsabilidad y está completamente desacoplado de los demás, de esta forma, si ocurre un cambio, se puede aislar en un solo componente sin afectar al resto, además, si un nuevo requerimiento es requerido, solo es necesario regresar un nuevo tipo de procesador de eventos que escuche un determinado tipo de evento.

Desventajas:

- Testabilidad: Una arquitectura distribuida y asíncrona agrega cierta complejidad a las pruebas, pues no es posible generar un evento y esperar un resultado para validar el resultado, en su lugar, es necesario crear pruebas más sofisticadas que creen los eventos y esperen la finalización del procesamiento del evento inicial más toda la serie

de eventos que se podrían generar como consecuencia del evento inicial para validar el resultado final.

- Desarrollo: Codificar soluciones asíncronas es complicado, pero aún más, es la necesidad de crear manejadores de errores más avanzados que permitan recuperarse en una arquitectura asíncrona, donde la falla en un procesador no significa la falla en el resto, lo que puede ocasionar la inconsistencia entre los sistemas.

Variantes:

Publish-Suscribe

- Componentes independientes publican eventos y otros se suscriben a ellos.
- Los publicantes ignoran el motivo global por el cual el evento es publicado, los suscriptores ignoran porque o quien publica el evento y dependen sólo del evento no de quien lo publica.
- Cada tópico puede tener más de un publicante y los publicantes pueden aparecer y desaparecer dinámicamente, lo que le da flexibilidad sobre configuraciones estáticas.
- Los suscriptores pueden suscribirse y des-suscribirse dinámicamente a un tópico.
- Aplica a elementos de tiempo de ejecución, es parte de la vista de ejecución (runtime)

Propiedades clave del patrón:

- Mensajería Muchos a muchos: los mensajes publicados son enviados a todos los suscriptores que están registrados en el tópico. Muchos publicantes pueden publicar en el mismo tópico, y muchos suscriptores pueden escuchar en el mismo tópico.
- Calidad de Servicio (QoS) Configurable: Además de mensajes seguros/no seguros, el mecanismo de comunicación subyacentes puede ser punto-a-punto o multicast/broadcast. El primero un mensaje distinto para cada suscriptor del tópico, el último manda un mensaje que todo suscriptor recibe.
- Bajo acoplamiento: No hay vínculo directo entre publicantes y suscriptores. Los publicantes no saben quien recibe su mensaje y los suscriptores no saben que publicante envía el mensaje.

Consideraciones:

- Arquitecturas muy flexibles y adecuadas para aplicaciones que requieren mensajes asíncronos uno-a-muchos, muchos-a-uno, muchos-a-muchos entre componentes

El principal beneficio es desacoplar los componentes que producen los eventos de quienes los consumen, lo que hace la arquitectura mantenible y evolucionable.

El bus de evento agrega una capa de indirección entre productores y consumidores, lo que podría dañar la performance del sistema. No obstante, existen recursos reusables que hacen ajustes a la performance.

Los canales (buses) de eventos varían en las propiedades que soportan:

- Durables: garantizan que cualquier mensaje que aceptan no se perderá durante una falla. Usualmente la durabilidad se logra escribiendo los eventos en un almacenamiento confiable, al menos temporalmente, lo que trae aparejada cierta latencia.
- Entrega en orden: garantizan entrega en orden o priorizada de eventos.
- Entrega en lotes: los eventos se acumulan en lotes para evitar cúmulos de eventos similares.

Broker

Propiedades clave del patrón:

- Arquitectura Hub-and-spoke: El agente actúa como concentrador de mensajes y los remitentes y receptores se conectan como rayos. Las conexiones al agente son por medio de puertos asociados a un formato específico de mensaje.
- Realiza ruteo de mensajes: el agente incrusta la lógica de procesamiento para entregar un mensaje recibido en un puerto de entrada en el puerto de salida. La entrega
- Realiza transformación de mensajes: el agente lógico transforma el tipo de mensaje de origen recibido en el puerto de entrada en el tipo de mensaje de destino requerido por el puerto de salida.

Consideraciones:

- Los agentes son adecuados para aplicaciones en las cuales los componentes intercambian mensajes que requieren grandes transformaciones entre los formatos de origen y destino
- El agente desacopla el remitente del receptor, permitiéndoles producir o consumir su formato nativo, y centraliza la definición de la transformación lógica en el agente para facilitar la comprensión y modificación.

Messaging

Esta arquitectura desacopla emisores de receptores utilizando una cola de mensajes intermedia. El emisor envía un mensaje al receptor y sabe que eventualmente será entregado, aunque la red esté caída o el receptor no esté disponible. Los cambios en los formatos de los mensajes de los receptores pueden causar cambios en las implementaciones de los emisores.

El patrón tiene inherentemente bajo acoplamiento, esto promueve alta modificabilidad, dado que emisores y receptores no se conectan directamente.

Propiedades clave del patrón:

- Comunicaciones asincrónicas: clientes envían pedidos a una cola, donde se almacenan hasta que una aplicación los remueve. Luego el cliente continúa sin esperar la respuesta.

- Calidad de Servicio (QoS) Configurable: la cola es configurable para una entrega de mayor velocidad, no confiable o de menor velocidad, confiable. Las operaciones de la cola pueden coordinarse con transacciones de base de datos.
- Bajo acoplamiento: no hay vínculo directo entre clientes y servidores. El cliente es inconsciente de cual servidor recibe el mensaje. El servidor es inconsciente de que cliente vino el mensaje.

Consideraciones:

- Las aplicaciones que pueden dividir el procesamiento de un pedido en pasos discretos, conectados por colas, tienen una extensión del patrón. Es el patrón Pipe & Filter.
- Provee una solución resistente para aplicaciones en las cuales la conectividad es transitoria, tanto debido a la red como a la poca confiabilidad de los servidores. Se puede implementar para pedido-respuesta sincrónico utilizando un par de colas pedido-respuesta.

Process Coordinator

Propiedades clave del patrón:

- Encapsulamiento de Proceso: El coordinador de proceso encapsula la secuencia de pasos necesarios para completar un proceso de negocio. La secuencia puede ser arbitrariamente compleja. El coordinador es un único de definición para el proceso de negocio, haciendo más fácil de comprender y modificar. Recibe un requerimiento de inicio de proceso, llama a los servidores en el orden definido por el proceso y emite los resultados.
- Bajo Acoplamiento: Los componentes del servidor son inconscientes de su rol en el proceso de negocio general, y del orden de los pasos del proceso. Los servidores simplemente definen un conjunto de servicios que pueden ejecutar y el coordinador los llama cuando es necesario como parte del proceso de negocio.
- Comunicaciones Flexibles: Las comunicaciones entre el coordinador y los servidores pueden ser sincrónicas o asincrónicas. Para las comunicaciones sincrónicas, el coordinador espera hasta que el servidor responda. Para las comunicaciones asincrónicas, el coordinador provee una llamada o respuesta cola/tópico, y espera hasta que el servidor responde utilizando

Consideraciones:

- El patrón coordinador de proceso es utilizado comúnmente para implementar procesos de negocio complejos que deben publicar requerimientos a algunos componentes servidores diferentes. Encapsulando la lógica de proceso en un lugar, es más fácil cambiar, administrar y monitorear la performance del proceso.

Patrones y estilos arquitectónicos

Los patrones están en una escala menor que los estilos. Múltiples patrones pueden aparecer en un mismo diseño. En contraste, un sistema usualmente tiene un único estilo arquitectónico dominante.

Los patrones y estilos a su vez se dividen en:

- Platónicos: idealizados, se los ve en los libros y rara vez de igual forma en el código.
- Embebidos: se los ve en sistemas reales y a menudo violan las restricciones estrictas de los platónicos. La violación generalmente implica una gran compensación.

Un estilo es una colección con nombre de decisiones de diseño arquitectónico que son aplicables en un contexto de desarrollo dado, restringen decisiones de diseño arquitectónico que son específicas de un sistema particular dentro de un contexto y obtienen cualidades beneficiosas en el sistema resultante.

- Estable un marco de referencia a partir del cual es posible construir aplicaciones que comparten un conjunto de atributos y características mediante las cuales es posible identificarlos y clasificarlos.
- Sirven para nombrar un diseño arquitectónico recurrente.
- No determinan tecnología ni detalles de Implementación.

Un patrón Expresa una organización estructural fundamental o esquema para sistemas de software. Proporciona un conjunto de subsistemas predefinidos, especifica sus responsabilidades e incluye reglas y pautas para organizar las relaciones entre ellos.

- Ayudan a definir características básicas y el comportamiento de una aplicación.
- Mejora la partición y promueve la reutilización del diseño, dando solución a problemas recurrentes.
- Comprende un conjunto de principios que dan forma a una aplicación.

Microservicios

Definicion

La arquitectura de microservicios es una SOA. Los servicios se conectan a través de interfaces proporcionadas por SOA.

Una aplicación se crea con componentes pequeños, independientes y poco acoplados, donde cada módulo actúa como una aplicación o servicio independiente.

Cada servicio puede funcionar solo, pero interactuar con otro módulo cuando sea necesario. Cada servicio en una aplicación de microservicio controla una función única con sus datos. Cada servicio en una aplicación basada en microservicios tiene una base de datos separada. Aunque esto puede causar datos redundantes, asegura un acoplamiento bajo. Cada servicio puede estar escrito en un lenguaje diferente y puede usar un tipo diferente de base de datos para satisfacer sus necesidades.

Micro proviene del hecho de que cada servicio debe enfocarse en hacer solo 1 tarea.

Se conectan entre sí a través de una interfaz de programación de aplicaciones (API), es decir, un conjunto de reglas estrictamente controladas que ambos pueden manejar. Estas colecciones o programas se pueden desarrollar, implementar y mantener de forma independiente, siempre que se garantice que funcionan bien con la API.

Los servicios generalmente se comunican usando algo como REST (potencialmente a través de una puerta de enlace API), están escritos en diferentes lenguajes y tienen almacenamientos independientes

Características:

- Composición vía servicios
Los servicios se despliegan independientemente.
Interfaces explícitas para evitar romper el encapsulamiento.
Las API remotas de grano grueso
- Productos no proyectos
Capacidades de negocio.
El producto puede utilizarse para asistir e incrementar las capacidades del negocio
El equipo es dueño de un producto durante todo su ciclo de vida, incluyendo la operación.
- Terminales inteligentes y tuberías bobas.
Microservicios lo más desacoplados y cohesivos posible.
Peticiones-respuestas HTTP con recursos API y mensajería liviana.
Utilizar un bus ligero de mensajes.
Cambiar el patrón de comunicaciones.
- Administración descentralizada.
Utilización de patrones, prácticas open source y código y librerías compartidas.
Evolución de los contratos de los servicios en forma independiente: Patrón Tolerant Reader y Consumer driven contracts.
Ética de: “el que lo construye, lo ejecuta”
- Gestión de datos descentralizada.
Persistencia políglota.
No coordinación de transacciones entre servicios
Operaciones compensatorias.
- Automatización de la infraestructura
Pruebas automatizadas
Despliegue automatizado e inversión en herramientas.
Automatización de la infraestructura
- Diseño para fallas.
Indisponibilidad del proveedor
Rastros para predicción de inconvenientes.

Detectar fallas y restaurar automáticamente.
Monitorización en tiempo real de la aplicación

Ventajas:

- Cada servicio es relativamente pequeño y fácil de entender para el desarrollador
- Más sencillo escalar el desarrollo.
- Mejora el aislamiento de las fallas.
- Elimina los compromisos de largo plazo con una tecnología determinada.
- Es posible liberar versiones más frecuentemente.

Desventajas:

- Crear un sistema distribuido es complejo
- Los IDE y herramientas
- Las pruebas distribuidas
- Mecanismos de comunicación inter servicios
- Transacciones distribuidas
- Se incrementa el consumo de memoria
- Performance: debido a la latencia, la comunicación por la red puede llegar a ser incluso más lenta que el proceso en sí.
- Múltiples puntos de falla: cada comunicación entre microservicios tiene una posibilidad de fallar.
- Trazabilidad: complica recuperar y realizar una traza completa de la ejecución de un proceso, pues cada Microservicio arroja de forma separada su traza o logs que luego deben de ser recopilados y unificados para tener una traza completa.
- Madurez del equipo de desarrollo: requiere de un equipo maduro de desarrollo y con un tamaño adecuado, pues los Microservicios agregan muchos componentes que deben ser administrados

Patrones

Hay 3 capas de patrones:

- Patrones de Infraestructura: aspectos por fuera del desarrollo. Tendencia a eliminar las máquinas físicas pesadas y de larga duración por tecnologías cada vez más ligeras y efímeras
- Aplicación de Infraestructura: aspectos de infraestructura que también impactan en el desarrollo.
- Patrones de Aplicación: problemas que enfrentan los desarrolladores

Patron de Descomposición por subdominio

Cada subdominio es mapeado a Servicio; cada servicio tiene su propio modelo de dominio, hay una traza directa; aplicando el patrón de Descomposición por subdominio.

Contexto Acotado (Bounded Context): Definición operacional donde un modelo particular es aplicable y está bien definido. (Comúnmente un subsistema).

Subdominio: parte de un dominio, basado en una descomposición conceptual particular del dominio.

Tipos de subdominio:

Core Domain (Dominio núcleo): Donde se realiza una inversión estratégica, prioritario. Distingue a la organización de la competencia.

Soporte: contribuyen al dominio core facilitando que se pueda llevar a cabo. Son necesarios y tienen sentido en relación con el core y en ese sentido son específicos.

Genérico: son aspectos que no son centrales del negocio, de hecho, son comunes a muchos tipos de negocio, y podrían incluso ser subcontratados, externalizados o comprados

Los servicios tienen que cumplir los siguientes criterios:

- Cada servicio tiene una única responsabilidad.
- Dividir la funcionalidad en dos servicios no debería resultar en una comunicación excesiva. Si es así, podría indicar que estas funciones pertenecen al mismo servicio.
- Cada servicio tiene un tamaño manejable, propicio para el desarrollo independiente por parte de un equipo pequeño.
- Ninguna interdependencia requiere la implementación de dos o más servicios simultáneamente.
- Los servicios están débilmente acoplados y pueden evolucionar de forma independiente.
- Los límites del servicio no comprometen la coherencia o integridad de los datos.
- Cuando no esté seguro, comience con microservicios más generales, ya que dividir un servicio en servicios más pequeños generalmente es más fácil que refactorizar la funcionalidad en los servicios existentes

Patron API Gateway

Una API Gateway rutea los requerimientos desde las aplicaciones móviles a los servicios.

Los servicios colaboran entre sí mediante APIs

API Gateway es responsable del enrutamiento de solicitudes, la composición y la traducción de protocolos. Proporciona a cada uno de los clientes de la aplicación una API personalizada.

API Gateway también puede enmascarar fallas en los servicios backend al devolver datos almacenados en caché o predeterminados.

La API Gateway cumple la función crucial de ocultar la ubicación y los detalles de implementación de los servicios vinculados a los endpoints de la API Gateway.

API Gateway de microservicios se abstiene de albergar lógica de negocios, orquestación o mediación, defendiendo el principio sagrado de un "contexto limitado" (Bounded context (DDD)).

La API Gateway maneja las solicitudes de dos maneras.

- Algunas solicitudes simplemente se envían/enrutan al servicio apropiado.
- Otras solicitudes las distribuye a múltiples servicios.

Ventajas:

- Aísla a los clientes de cómo se divide la aplicación en microservicios.
- Aísla a los clientes del problema de determinar las ubicaciones de las instancias de servicio.
- Proporciona la API óptima para cada cliente.
- Reduce el número de solicitudes/ida y vuelta.
- Menos solicitudes también significan menos gastos generales y mejoran la experiencia del usuario.
- Una API Gateway es esencial para las aplicaciones móviles.
- Simplifica el cliente al mover la lógica para llamar a múltiples servicios desde el cliente a la API Gateway.
- Se traduce desde un protocolo API público "estándar" compatible con la web a cualquier protocolo que se utilice internamente.

Desventajas:

- Mayor complejidad: la API Gateway es otra parte que debe desarrollarse, implementarse y gestionarse.
- Mayor tiempo de respuesta: debido al salto de red adicional a través de la API Gateway

Patron backends for frontends

Define una API Gateway separada para cada clase de cliente

Ventajas:

- Seguridad: cuando necesita asegurarse de que sus AP's protejan los datos confidenciales de manera confiable
- Escalabilidad: cuando necesita mejorar el rendimiento de su API mediante la implementación de funciones de rendimiento, como gestión del tráfico, almacenamiento en caché de datos, etc.
- Interoperabilidad: cuando dedica demasiado tiempo a las necesidades de mediación y transformación de datos para solicitudes entrantes y salientes para admitir diferentes formatos y requisitos de datos.
- Monitoreo y registro: cuando necesita una cabina central para el registro, auditoría y monitoreo de sus API's.
- Monetización: cuando necesita una manera sencilla de empaquetar, fijar precios, promocionar y monitorear sus API durante todo su ciclo de vida

Patron una base de datos por servicio

Cada microservicio debe manejar sus datos en forma privada y sólo acceder a ellos a través de su API. La base de datos del servicio es parte efectiva de la implementación del servicio. No puede accederse directamente por otros servicios.

Se requiere actualizar continuamente el estado del pedido considerando los procesos que suceden en el flujo del pedido. Los servicios están conectados a través de múltiples fuentes de datos. Asegurarse de actualizar los datos precisamente para completar un pedido con éxito.

Mantener ACID: para garantizar la precisión de una transacción, debe cumplir con los principios de Atomicidad, Consistencia, Aislamiento y Durabilidad (ACID).

- La atomicidad garantiza que se completen todos los pasos de una transacción o ninguno.
- La consistencia garantiza la transición de datos de un estado válido a otro.
- El aislamiento garantiza que las transacciones simultáneas produzcan los mismos resultados que las transacciones secuenciales.
- La durabilidad garantiza que las transacciones comprometidas no se vean afectadas por fallas del sistema

Gestionar el nivel de aislamiento de transacciones: define el alcance de la visibilidad de los datos en una transacción cuando se produce el acceso simultáneo de otros servicios. Dicho de otra manera, cuando un microservicio mantiene un objeto en la base de datos mientras otra solicitud lee los datos, surge la pregunta: ¿debería el servicio proporcionar los datos antiguos o actualizados?

Ventajas:

- Bajo acoplamiento.
- Utilizar la base de datos más conveniente.

Desventajas:

- Implementación compleja.
- Complejidad de manejar Bases de datos SQL y NoSQL.

Patron single service per host

Contexto: Ha aplicado el patrón de arquitectura de microservicio y ha diseñado su sistema como un conjunto de servicios. Cada servicio se despliega como un conjunto de instancias de servicio para lograr rendimiento y disponibilidad

Motivaciones:

- Los servicios se escriben utilizando una variedad de lenguajes, marcos y versiones de marcos.

- Cada servicio consta de múltiples instancias de servicio para rendimiento y disponibilidad.
- Escalar e implementar el servicio de forma independiente.
- Aislar las instancias de servicio unas de otras
- Crear e implementar rápidamente un servicio.
- Poder limitar los recursos (CPU y memoria) consumidos por un servicio.
- Monitorear el comportamiento de cada instancia de servicio.
- Que la implementación sea confiable
- Implementar la aplicación de la manera más rentable posible

Despliegue cada instancia de servicio única en su propio host.

Ventajas:

- Las instancias de servicios están aisladas unas de otras.
- No hay posibilidad de que entren en conflicto requisitos de recursos o versiones de dependencia.
- Una instancia de servicio solo puede consumir como máximo los recursos de un único host
- Es sencillo monitorear, administrar y volver a implementar cada instancia de servicio

Desventajas:

- Utilización de recursos potencialmente menos eficiente en comparación con servicios múltiples por host porque hay más hosts

Existen 2 variantes:

- Service Instance per Virtual Machine
- Service Instance per Container

Patron multiple service per host

Motivaciones

- Los servicios se escriben utilizando una variedad de lenguajes, marcos y versiones de marcos.
- Cada servicio consta de múltiples instancias de servicio para rendimiento y disponibilidad.
- Escalar e implementar el servicio de forma independiente.
- Aislar las instancias de servicio unas de otras
- Crear e implementar rápidamente un servicio.
- Poder limitar los recursos (CPU y memoria) consumidos por un servicio.
- Monitorear el comportamiento de cada instancia de servicio.
- Que la implementación sea confiable

- Implementar la aplicación de la manera más rentable posible.

Contexto: Ha aplicado el patrón de arquitectura de microservicio y ha diseñado su sistema como un conjunto de servicios. Cada servicio se implementa como un conjunto de instancias de servicio para lograr rendimiento y disponibilidad

Ejecute múltiples instancias de diferentes servicios en un host (máquina física o virtual). Existen varias formas de implementar una instancia de servicio en un host compartido, entre las que se incluyen:

- Implemente cada instancia de servicio como un proceso JVM. Por ejemplo, instancias de Tomcat o Jetty por instancia de servicio.
- Implemente varias instancias de servicio en la misma JVM. Por ejemplo, como aplicaciones web o paquetes OSGI.

Ventajas:

- Utilización de recursos más eficiente que el patrón una instancia de servicio por host.
- Es aún más eficiente si un proceso o grupo de procesos ejecuta múltiples instancias de servicio, por ejemplo, múltiples aplicaciones web que comparten el mismo servidor Apache Tomcat y JVM.
- La implementación de un servicio es relativamente rápida

Desventajas

- Riesgo de requisitos de recursos contradictorios.
- Riesgo de versiones de dependencia conflictivas.
- Difícil limitar los recursos consumidos por una instancia de servicio.
- Si se implementan varias instancias de servicios en el mismo proceso, entonces es difícil monitorear el consumo de recursos de cada instancia de servicio.
- Es imposible aislar cada instancia

POO

Realización: Es una relación que especifica que una clase es la encargada de implementar el comportamiento especificado en otra.

Una interfaz es una Pieza de código que tiene dos roles:

- Definir comportamiento que aplica a múltiples tipos.
- Hacer foco principal en las clases que usan esos tipos.

La interfaz esta estructurada como una clase abstracta la cual solo tiene metodos abstractos, es decir sin codigo, todos los metodos de la interfaz deben estar implementados en sus hijos.

Las interfaces se usan cuando el cliente se comunica a través de peticiones a la clase de realización y en tiempo de ejecución se decide a qué clase hija usa.

Hay 2 tipos de interfaces:

- Provista: ofrece los servicios de la interfaz
- Requerida: utiliza los servicios de la interfaz provista

Estos 2 tipos están asociadas a un servicio o clase que tengan características concretas

Program to Interface Principle

- Use interfaces si tiene múltiples implementaciones concretas.
- Utilice interfaces (el principio de diseño) para desacoplar su propio sistema de un sistema externo.

Principio de Inversión de Dependencias

Normalmente la capa de lógica de negocio se comunica con la capa de persistencia, este principio sirve para mantener la SOC, lo que dice el principio es que la capa de persistencia se comunique con las entidades de la capa de lógica de negocio

- Los módulos de alto nivel no deben depender de los módulos de menor nivel
- Ambos deben depender de sus abstracciones
- Las abstracciones no deben depender de los detalles.